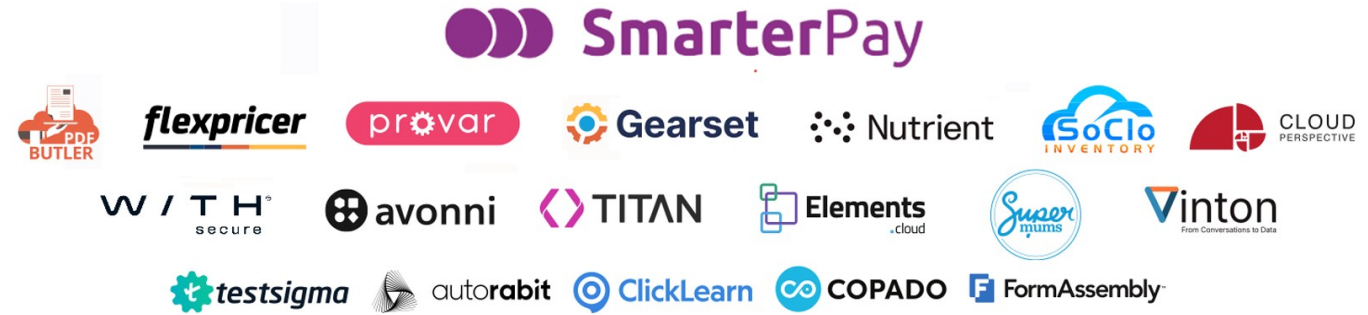




Scaling Apex and LWC with Apex Cursors

Andrew Fawcett, AndyInTheCloud Consulting Ltd

andy@andyinthecloud.com

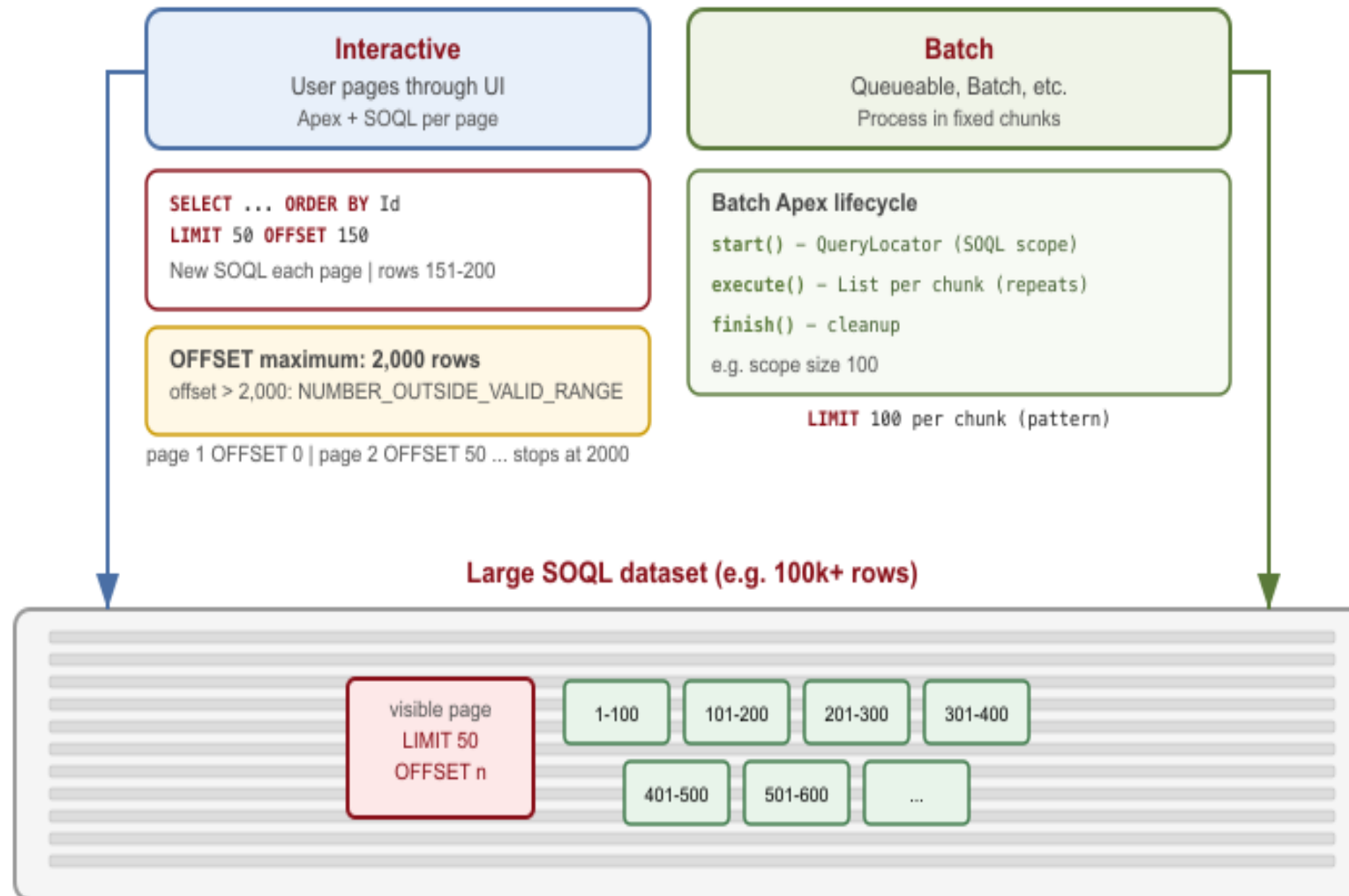
<https://www.linkedin.com/in/andyfawcett/>

#LDNsCall #LC26



Accessing and processing datasets today in SOQL

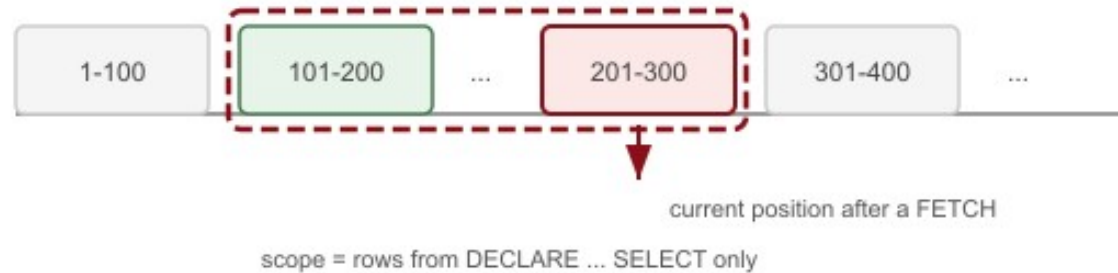
UI paging with OFFSET/LIMIT vs async processing in fixed-size chunks.



What is a SQL Cursor?

Traverse a query result incrementally instead of returning the full set at once.

1. One declared result set - position moves along rows



Sections 2 and 3 = two views of the same idea

2 = lifecycle statements (DECLARE ... CLOSE)

3 = access patterns (how FETCH moves in that scope)

Different from OFFSET paging (new query per page)

2. Lifecycle pattern (SQL statements)

Ordered steps the application executes

- 1 **DECLARE** c **CURSOR FOR** <select>;
Defines query scope
- 2 **OPEN** c;
Before first row
- 3 **FETCH** ... **FROM** c **INTO** ...; (repeat)
NEXT, PRIOR, ABSOLUTE...
- 4 **CLOSE** c;
Release resources
- 5 **DEALLOCATE** c; (some products)
Drop definition

Example

```
DECLARE account_cur CURSOR FOR
SELECT id, name FROM accounts
WHERE active ORDER BY id;

OPEN account_cur;
FETCH NEXT FROM account_cur ...
CLOSE account_cur;
```

3. Access patterns (how you read rows)

FETCH orientations - same open cursor, move position in scope

Forward-only cursor (typical default)

FETCH NEXT - one row forward
Most common in ETL / streaming loops

Scrollable cursor (when supported)

FETCH NEXT | **PRIOR** | **FIRST** | **LAST**

FETCH ABSOLUTE n | **FETCH RELATIVE** n

Jump relative to start or current position (product-dependent)

Not a cursor access pattern:

SELECT ... OFFSET 20 FETCH 10

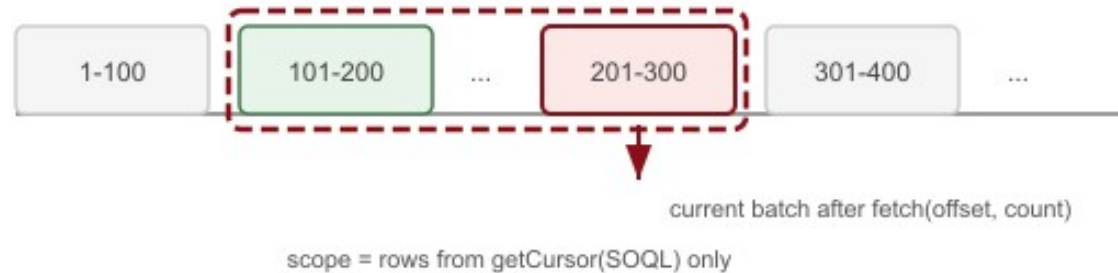
new query each page - no held position
cursor: one OPEN, many FETCH in same scope



What is a SOQL Cursor?

SQL cursor concepts on Salesforce: Database.getCursor(SOQL) and incremental fetch in Apex.

1. One SOQL result set - position moves along rows



Apex maps SQL cursor ideas to API calls

2 = lifecycle (getCursor, fetch, hold state)

3 = access patterns (offset batch vs fetchPage)

Not [SELECT ...] loading all rows at once

2. Lifecycle pattern (Apex API)

No DECLARE/OPEN/CLOSE - platform cursor over SOQL

- 1 Database.getCursor(soql);
Or PaginationCursor
- 2 cursor.getNumRecords();
Optional row count
- 3 cursor.fetch(offset, count);
Repeat until done
- 4 Hold cursor: LWC or Cache
Available across requests
- 5 Governed by Limits.*ApexCursor*
Apex cursor limits

Example (this repo)

```
Database.Cursor c =
    Database.getCursor(
        'SELECT Id, Name FROM Account
        WHERE Name LIKE 'TEST%'
        ORDER BY Name',
        AccessLevel.USER_MODE);
List<Account> batch =
    c.fetch(offset, pageSize);
```

3. Access patterns (how you read rows)

Incremental reads inside one open SOQL scope

Database.Cursor (standard)

fetch(offset, count)
Forward by offset into scope (infinite scroll)

Database.PaginationCursor (GA)

fetchPage(start, pageSize)
Page-oriented access; CursorFetchResult

Not a SOQL cursor pattern:

List<SObject> = [SELECT ...]

position in scope

0-99 100-199 ... 200-299

fetch(200, 100) = rows 200-299

all rows in heap at once
cursor: one SOQL scope, many fetches

Usecases for Apex Cursors

Interactive

- Analyst large data view
- Reporting and drilldown
- Preview job records to process

```
apex>  
Database.PaginationCursor c =  
    Database.getPaginationCursor(  
        soql, AccessLevel.USER_MODE);  
Database.CursorFetchResult r =  
    c.fetchPage(start, pageSize);
```

Batch

- Orchestrate process execution
- Look ahead to optimize resource usage
- Safer immutable record set

```
apex>  
Database.Cursor c =  
    Database.getCursor(  
        soql, AccessLevel.USER_MODE);  
List<Account> batch =  
    c.fetch(offset, pageSize);
```

Architecture

24hr cached large record sets | Scalable replacement to SOQL OFFSET | Bidirectional navigation

Demo 1: Virtual Data Table



Apex Cursor Demo

Infinite Loading

Reporting Drilling Down

Adaptive Async

Infinite Loading

Standard Cursor

Pagination Cursor

☒ Use Session Cache

Refresh

	Account Name	Industry	Type	Billing City	Phone
1	TEST_Acme_Alliance_00234_1779980748800	Hospitality	Other	TEST City 34	+1-555-334-1234
2	TEST_Acme_Alliance_00494_1779980749060	Hospitality	Other	TEST City 94	+1-555-594-1494
3	TEST_Acme_Alliance_00754_1779980749320	Hospitality	Other	TEST City 54	+1-555-854-1754
4	TEST_Acme_Alliance_01014_1779980749580	Hospitality	Other	TEST City 14	+1-555-214-2014
5	TEST_Acme_Alliance_01274_1779980749840	Hospitality	Other	TEST City 74	+1-555-474-2274
6	TEST_Acme_Alliance_01534_1779980750100	Hospitality	Other	TEST City 34	+1-555-734-2534
7	TEST_Acme_Alliance_01794_1779980750360	Hospitality	Other	TEST City 94	+1-555-994-2794

Debug Info

Loaded Records: 50
Total Records: 5000
Has More: true
Cursor Type: Pagination Cursor
Using Session Cache: true
Deleted Rows Skipped: 0

Limits Info

TRANSACTION
Standard Cursors: 0/50
Standard Cursor Rows: 0/50000000
Pagination Cursors: 0/50
Pagination Cursor Rows: 0/100000
Fetch Calls: 1/100

DAILY (ORG)
Daily Standard Cursors: 2/10000
Daily Pagination Cursors: 5/200000
Daily Cursor Rows: 35000/100000000

Demo 1: Virtual Data Table Summary

1

```
apex>  
Database.Cursor c =  
Database.getCursor(  
    soql,  
    AccessLevel.USER_MODE);
```

2

```
apex>  
List<Account> batch =  
    c.fetch(offset,  
            batchSize);  
result.hasMore =  
    offset < c.getNumRecords();
```

3

```
apex>  
const result = await  
loadMoreRecords({  
    request, useSessionCache  
});
```

- Infinite scroll loads 50 rows at a time — no SOQL OFFSET
- Cursor held in LWC state or Cache.Session across Apex calls
- Live limit panel surfaces cursor, fetch, and row governor usage
- Switch between standard and pagination cursor modes in one LWC

Demo 2: Interactive Reporting and Drilldown



Demo 2: Interactive Reporting and Drilldown Summary

1

```
apex>
Database.PaginationCursor pc =
  Database.getPaginationCursor(
    soql,
    AccessLevel.USER_MODE);
Cache.Session.put(
  sessionKey(product), pc);
```

2

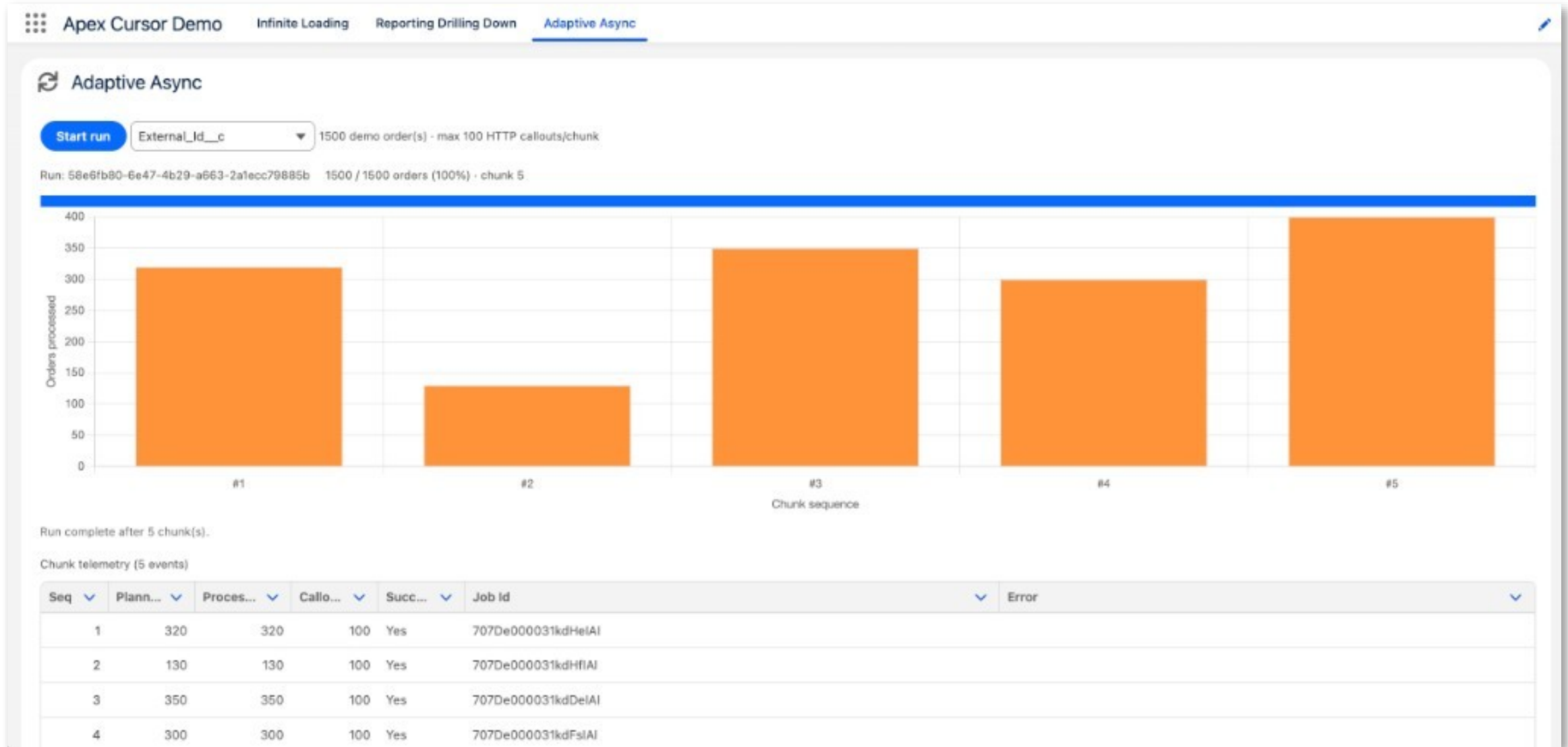
```
apex>
fetchResult = pc.fetchPage(
  dayIndex, 1);
deletedRows =
  fetchResult
    .getNumDeletedRecords();
```

3

```
apex>
const detail = await
  fetchDetailRange({
    product,
    startIndex, endIndex
  });
```

- PaginationCursor builds the monthly chart with one fetchPage per month
- Session cache keeps the cursor for brush-selection drill-down
- Index-based fetchPage maps chart range to daily observation rows
- fetchPage reports deleted rows when underlying data changes

Demo 3: Adaptive Async



Demo 3: Adaptive Async Summary

1

```
apex>  
Database.Cursor cursor =  
    Database.getCursor(  
        soql,  
        AccessLevel.USER_MODE);  
System.enqueueJob(  
    new AdaptiveOrderWorkerQueueable(  
        runId, cursor, 1), options);
```

2

```
apex>  
for (Order row :  
    cursor.fetch(pos,  
        batchSize)) {  
    rowCallouts =  
        rowCalloutsFor(row);  
    // pack to callout budget
```

3

```
apex>  
if (moreOrders) {  
    sequence++;  
    System.enqueueJob(this);  
}  
System.attachFinalizer(this);
```

- One cursor opened in Apex and passed through chained queueable jobs
- Chunk size adapts to remaining HTTP callout budget per transaction
- Finalizer publishes platform events after each chunk commits
- Cursor position travels on the job — no Database.Stateful required

Usage Best Practices

Stable ORDER BY

```
apex>  
'SELECT ... FROM Account  
  WHERE Name LIKE 'TEST%'  
  ORDER BY Name',  
  AccessLevel.USER_MODE);
```

Sort on a unique, stable key.
Keeps offsets and page indexes predictable.

Retry transient errors

```
apex>  
{ catch (System.  
  TransientCursorException e) {  
    if (++retries <= MAX)  
      return retry();
```

TransientCursorException may be retried.
Do not retry FatalCursorException.

Guard fetch size

```
apex>  
remaining = c.getNumRecords()  
  - offset;  
fetchSize = remaining > 0  
  ? Math.min(batchSize,
```

Skip fetch when remaining is zero.
Avoids empty fetch(0, 0) batches.

Cursor returns to LWC

```
apex>  
@AuraEnabled  
public static Result load(...) {  
  result.cursor = cursor;  
  return result;
```

Cursors are Aura-serializable.
Return on @AuraEnabled result — no JSON
hack.

Cap fetchPage size

```
apex>  
pageSize = Math.min(  
  PAGE_SIZE,  
  pc.getNumRecords()  
  - startIndex);
```

Do not request rows past the result set end.
Critical for partial last pages and after deletes.

Surface limits in UI

```
apex>  
Limits.getApexCursors()  
  + '/' + Limits  
  .getLimitApexCursors();  
OrgLimits.getMap().get(
```

Limits.* for per-transaction governors.
OrgLimits.getMap() for rolling 24-hour org caps.

Limits



Limit	Pagination	Standard
Per transaction (each Apex request)		
Cursor instances open	50	50
Rows across all cursors in txn	100,000	50,000,000
Fetch calls (shared)	100	100
Per pagination fetchPage call		
Max rows per page	2,000	—
Per org (rolling 24 hours)		
Cursor instances opened	200,000	10,000
New rows fetched (shared)	100,000,000	100,000,000

- Transaction limits apply to each Apex request; daily limits roll per org over 24 hours.
- Fetch calls and daily row counts are shared across standard and pagination cursors.
- Cursors expire like API query cursors — see API Query Cursor Limits in Salesforce Help.

Tips and Tricks

Practical patterns from the Apex Cursor demos



Instrument limits first

Show cursor, fetch, and row limits in the UI before users hit governor walls.



Pagination for drill-down

PaginationCursor + fetchPage for deleted rows and chart index ranges.



Right-size each fetch

Smaller batches feel snappier; larger batches mean fewer fetch calls.



Session cache vs LWC state

Cache.Session survives Apex calls; LWC-held cursor avoids server-side state.



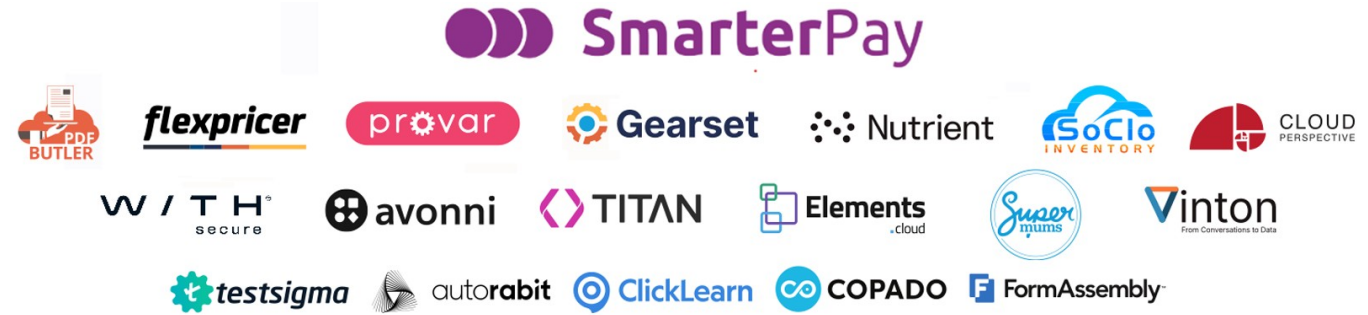
Stable ORDER BY

Sort on a unique key so offsets and page indexes stay predictable.



USER_MODE by default

Pass AccessLevel.USER_MODE on getCursor unless elevated access is required.



Q&A



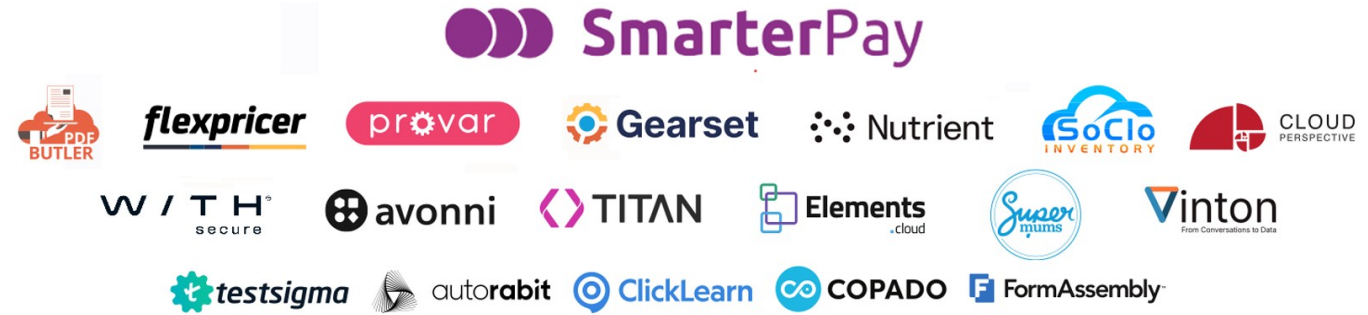
#LDNsCall #LC26

Andrew Fawcett

andy@andyinthecloud.com | LinkedIn

[View Session Resources](#)





Thank You

Andrew Fawcett

andy@andyinthecloud.com | LinkedIn

#LDNsCall #LC26

